

Ministry of Education and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #7

**Actuators – Actuation and Power Conversion. Part 1:
Binary Actuator Control**

Author:

Alexandru Cebotari

Group FAF-233

Verified by:

Alexei Martiniuc

Technical task and scope

This report documents only Part 1 of the actuator laboratory assignment. The developed application controls one binary actuator from STDIO and hardware input, applies software debouncing, and reports the current actuator state on LCD and serial output.

The current implementation uses:

- one **external LED** on D12 as the binary actuator accepted for the laboratory demonstration;
- one **pushbutton** on D2 as the binary hardware command source;
- one **LCD1602 in 4-bit mode** for displaying the binary actuator state and companion runtime data;
- one **serial STDIO interface** for ON/OFF commands and structured reports;
- one **cooperative scheduler** with separate tasks for commands, inputs, conditioning, actuation, and reporting.

The report intentionally focuses on Part 1 only. It does not present Part 2 as the scope of Lab 7.

Objectives

- Implement binary actuator control through serial commands.
- Apply software debouncing and persistent state validation for the binary command path.
- Allow hardware interaction through a pushbutton input.
- Display the actuator state and associated alerts on an LCD.
- Print a structured binary-actuator report through STDIO.
- Keep the project modular, PlatformIO-compatible, and ready for Wokwi simulation.

1 1. Domain analysis

1.1 1.1 Technological context

Binary actuator control is one of the simplest but most important hardware-software interaction patterns in embedded systems. A binary actuator has only two states, ON and OFF, but the command path that drives it still requires validation, filtering, and stable

reporting. In practice, noisy transitions from buttons or rapidly repeated commands can cause false switching if the command is applied directly.

For this reason, the binary actuator is not controlled by simple pin mirroring. Instead, the application separates command interpretation, debouncing, actuation, and reporting. This follows the guidance from the actuator chapter and the report documentation requirements.

1.2 1.2 Selected components and technologies

- Hardware: Arduino Uno, external LED with resistor, pushbutton, LCD1602 16-pin parallel interface, breadboard, jumper wires.
- Development tools: PlatformIO, Arduino framework, Wokwi, LaTeX, PlantUML.
- Libraries: LiquidCrystal for the LCD and the Arduino standard library.

The external LED is used as the binary actuator for the current laboratory demonstration. The assignment text allows another agreed binary actuator, while the practical guide also treats LEDs as valid binary actuator examples in embedded applications. This makes the setup simple, safe, and easy to validate in Wokwi and on real hardware.

1.3 1.3 Existing-solution reasoning and chosen interpretation

The strict textbook example for Part 1 is a relay-controlled lighting load. However, the laboratory also allows another binary actuator agreed with the mentor, and the guide explicitly discusses LEDs as binary ON/OFF devices. For the current implementation, the LED is used as the binary actuator because it is available in the kit, easy to simulate, and sufficient for demonstrating the required command-conditioning and reporting behavior.

Compared with a direct LED-blink demo, the developed application is stronger because it includes a command interpreter, a debouncing stage, periodic actuation, and a reporting interface. Therefore, the focus remains on the binary control architecture rather than on the electrical power stage.

1.4 1.4 Problem definition

The application must receive ON/OFF control commands through STDIO, optionally accept hardware interaction through a pushbutton, debounce the binary command state, apply the resulting stable state to the actuator, show the current state on the LCD, and print a structured status report on the serial interface. The implementation must remain modular and must satisfy the response-latency expectation under 100 ms.

Section conclusion

The selected hardware and software stack directly matches the Part 1 binary-actuator theme and provides enough visibility to document command interpretation, conditioning, display, and validation in a methodical way.

2. Conceptualization and design

2.1 Technical requirements

ID	Requirement	Type	Acceptance target
R1	The application shall receive binary ON/OFF commands through STDIO.	F	Valid serial commands are recognized
R2	The application shall drive one binary actuator.	F	Actuator state changes according to stable command
R3	The application shall apply software debouncing to the binary command path.	F	No false rapid switching
R4	The application shall support a binary hardware input through a pushbutton.	F	Button input can control the actuator in hardware-enabled modes
R5	The application shall show current actuator state on the LCD.	F	LCD exposes the binary output state
R6	The application shall print structured binary-actuator reports through STDIO.	F	Serial report is readable and periodic
R7	The system shall keep a modular software structure.	NF	app / tasks / services / drivers remain separated
R8	The system shall respond to command changes in less than 100 ms.	NF	Nominal debounce-based reaction remains below 100 ms
R9	The solution shall build in PlatformIO and run in Wokwi on Arduino Uno.	NF	Successful build and simulation resources exist

Table 1: Requirements matrix for Lab 7 Part 1

2.2 Validation criteria and traceability

Req.	Design anchor	Test IDs	Expected proof
R1	Command parser + command task	T1	Serial ON/OFF commands are accepted
R2	Indicator LED driver + actuator task	T2	LED state follows the stable command
R3	Binary conditioning flow	T3	Rapid transitions do not cause false toggling
R4	Hardware input driver + input task	T4	Pushbutton can request state changes
R5	LCD driver + reporting task	T5	LCD shows binary output state
R6	Reporting task + serial bridge	T6	Structured serial report is visible
R7	Source tree organization	T7	Modular directory structure is evident
R8	20 ms task period + debounce window	T8	Stable response under 100 ms
R9	PlatformIO + Wokwi descriptors	T9	Build succeeds and simulation resources exist

Table 2: Requirement-to-test traceability

2.3 Architectural design

The implementation is divided into four logical layers:

- **Application layer:** scheduler and shared runtime state.
- **Task layer:** command handling, hardware input sampling, conditioning, actuation, and reporting.
- **Service layer:** command parsing, binary conditioning, and runtime telemetry.
- **Driver layer:** LED actuator, LCD, serial bridge, and pushbutton input.

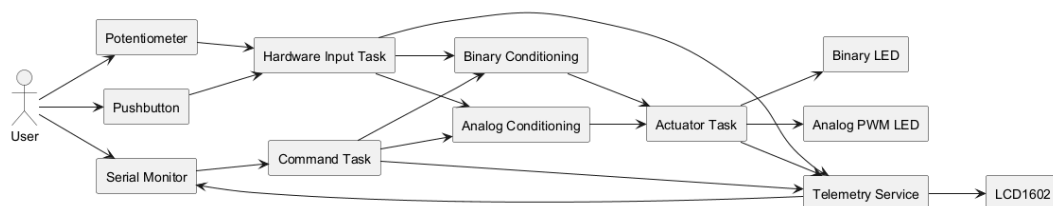


Figure 1: System structure for the binary actuator application

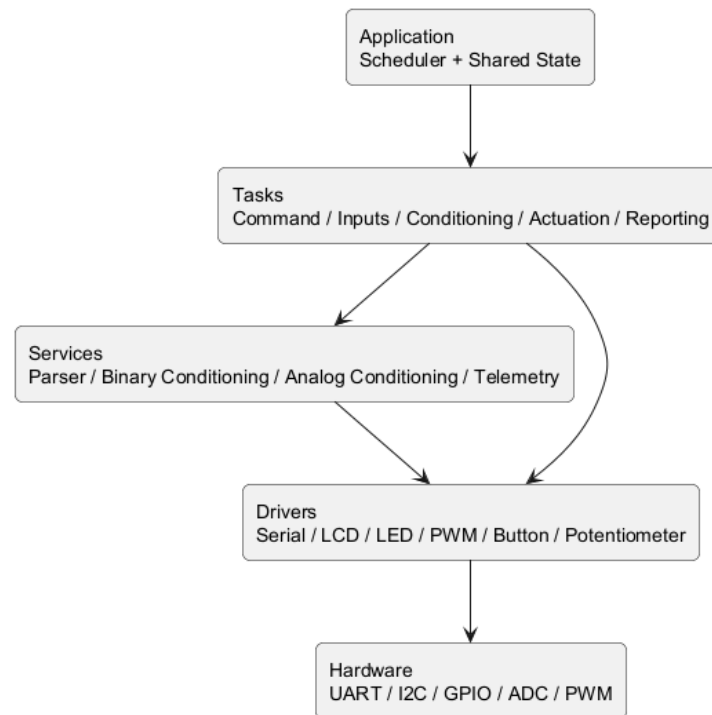


Figure 2: Hardware-software interface layers

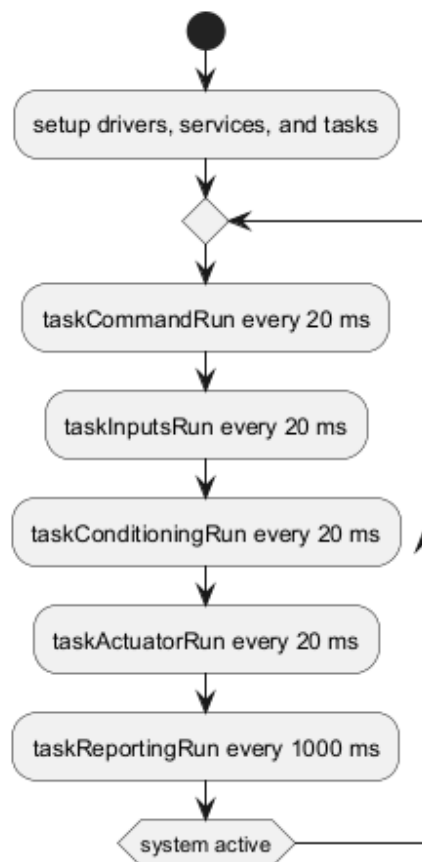


Figure 3: Cooperative scheduler used by the application

These diagrams show the structure required by the documentation guide: a system view, a HW/SW interface view, and the execution backbone used for the software.

2.4 Behavioral modeling

The binary actuator application is centered on three functional flows:

- interpretation of ON/OFF commands from serial or hardware sources;
- binary conditioning through temporal debouncing and stable-state validation;
- application of the stable actuator state followed by LCD and serial reporting.

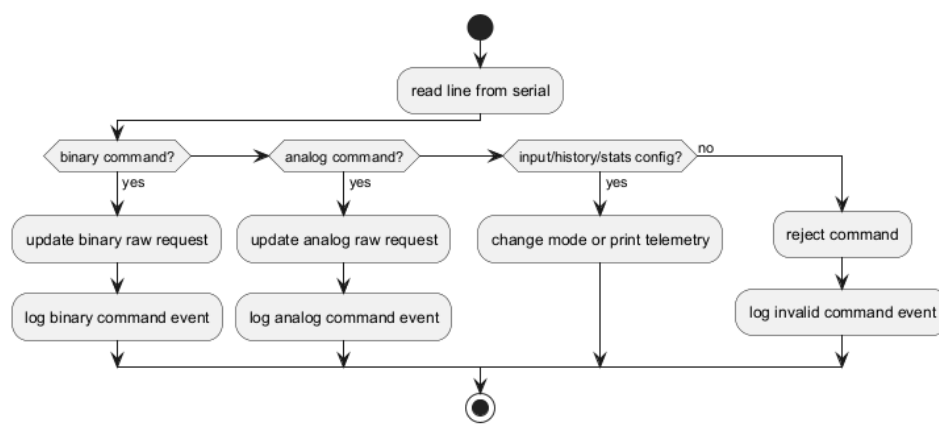


Figure 4: Command flow for the binary actuator application

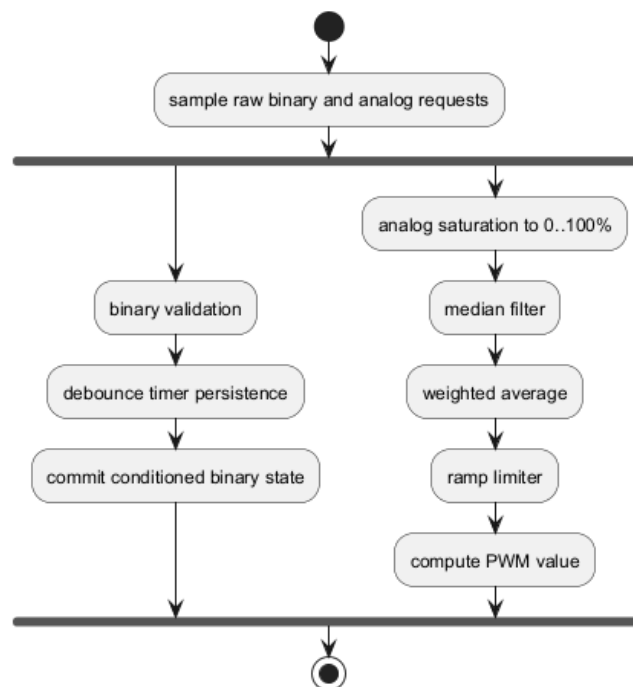


Figure 5: Binary command conditioning flow

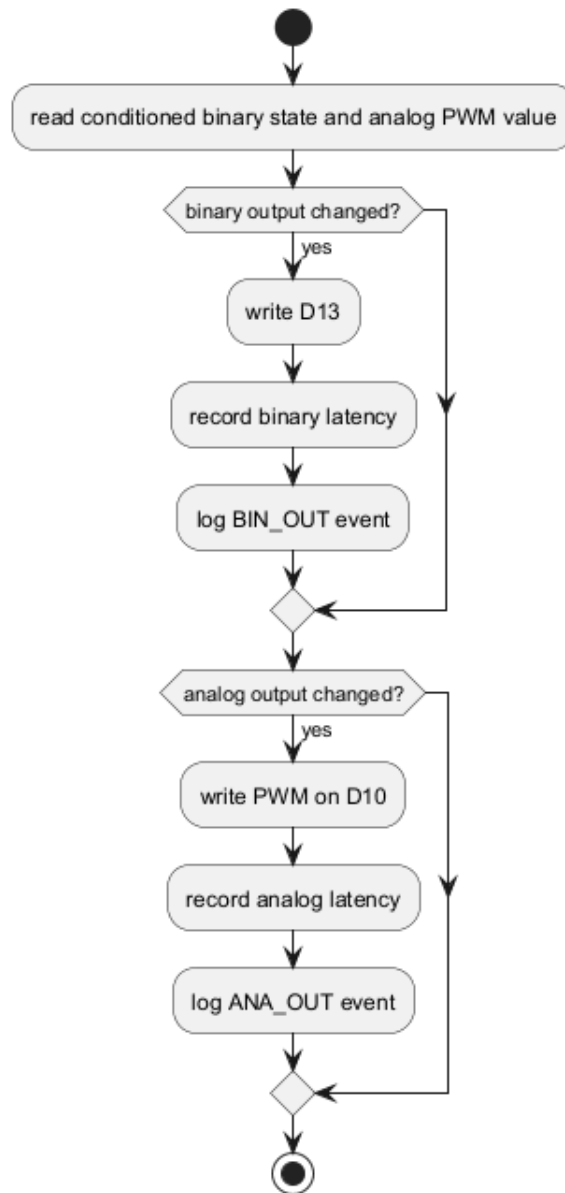


Figure 6: Actuator update flow

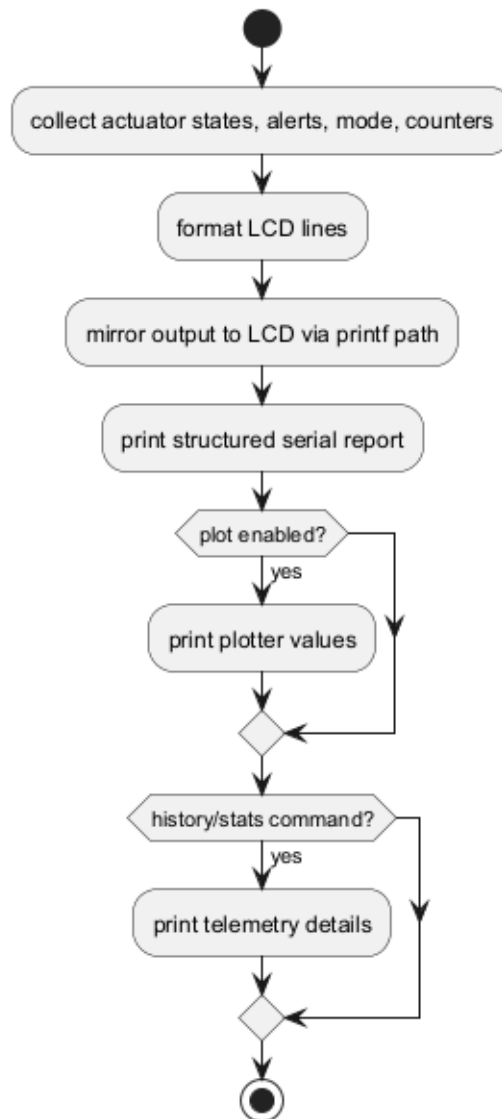


Figure 7: LCD and STDIO reporting flow

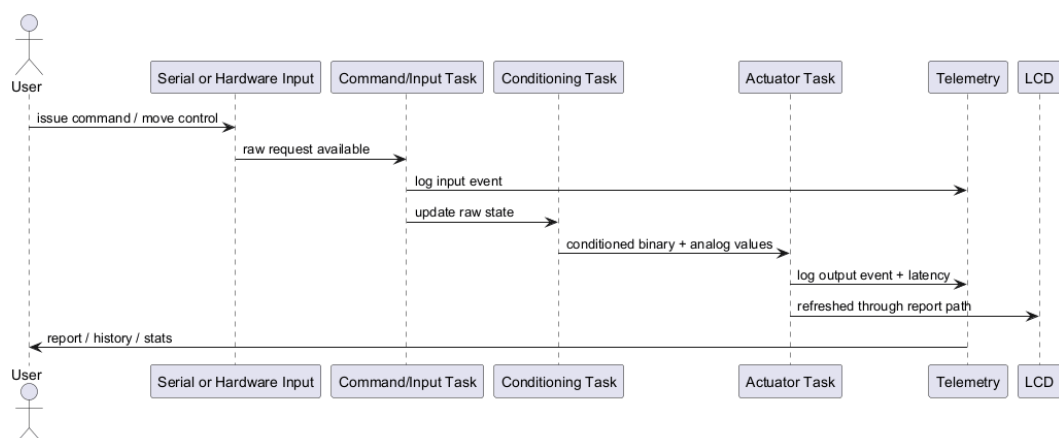


Figure 8: Runtime sequence from command to actuator output

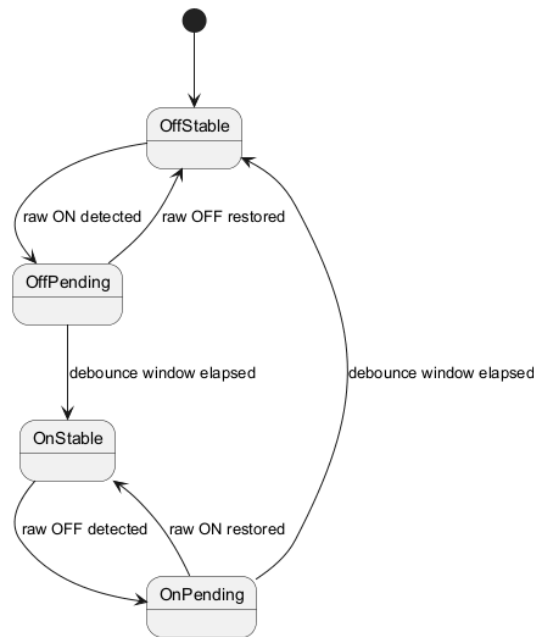


Figure 9: Binary actuator state machine

The behavioral diagrams cover the command path, the binary conditioning logic, the reporting path, and the state transitions of the binary actuator, which satisfies the diagram expectations from the project guideline.

2.5 2.5 Test scenarios

ID	Scenario	Req.	Pass criterion
T1	Send ON/OFF commands through STDIO.	R1	Commands are accepted and interpreted correctly
T2	Observe the LED actuator after a valid command.	R2	Binary actuator changes state correctly
T3	Toggle the input rapidly around a transition.	R3	Debouncing prevents unstable toggling
T4	Press the pushbutton in hardware-enabled mode.	R4	Pushbutton can request actuator state change
T5	Observe the LCD during runtime.	R5	LCD shows the binary output state
T6	Observe the serial report output.	R6	Structured report is printed periodically
T7	Inspect the source tree and module boundaries.	R7	Responsibilities remain separated
T8	Measure debounce-based reaction timing.	R8	Stable reaction remains under 100 ms
T9	Run <code>pio run</code> and inspect simulation files.	R9	Build succeeds and Wokwi resources exist

Table 3: Validation test plan for Lab 7 Part 1

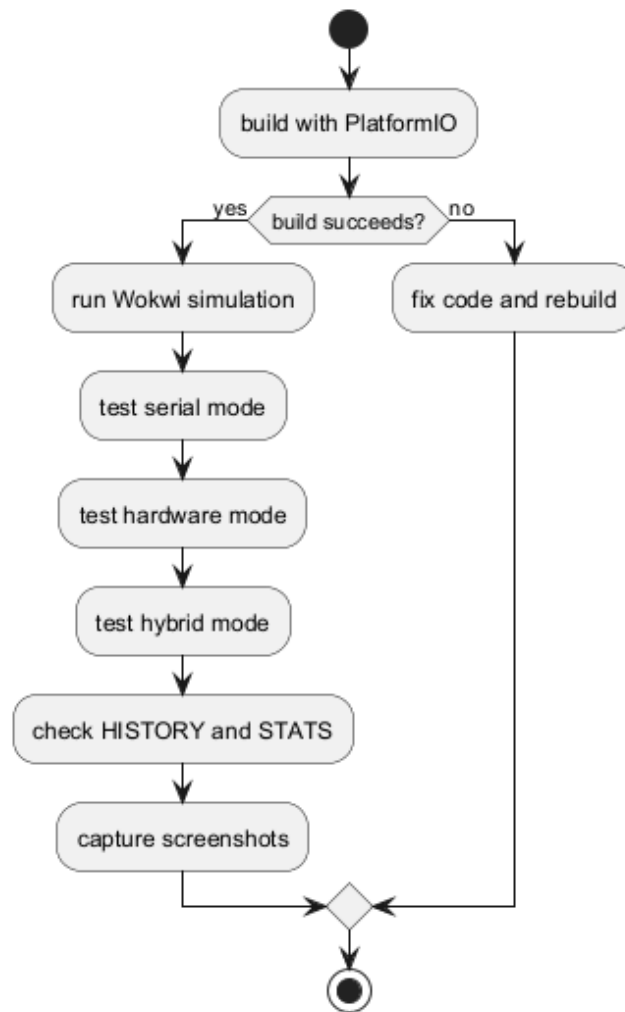


Figure 10: Validation workflow

Section conclusion

The design phase translates the Part 1 statement into explicit requirements, architectural views, behavioral diagrams, and test scenarios that can be validated one by one.

3 3. Hardware and software implementation

3.1 3.1 Implemented hardware configuration

The final `Code/diagram.json` and the real-hardware adaptation use the following practical wiring:

- binary actuator LED on D12;
- pushbutton input on D2;
- LCD1602 in 4-bit mode on D3-D7 and D13;

- serial STDIO through USB at 115200 baud.

This setup focuses on the binary actuator behavior required by Part 1. The LED is used as the implemented binary actuator, while the pushbutton provides a physical binary command source.

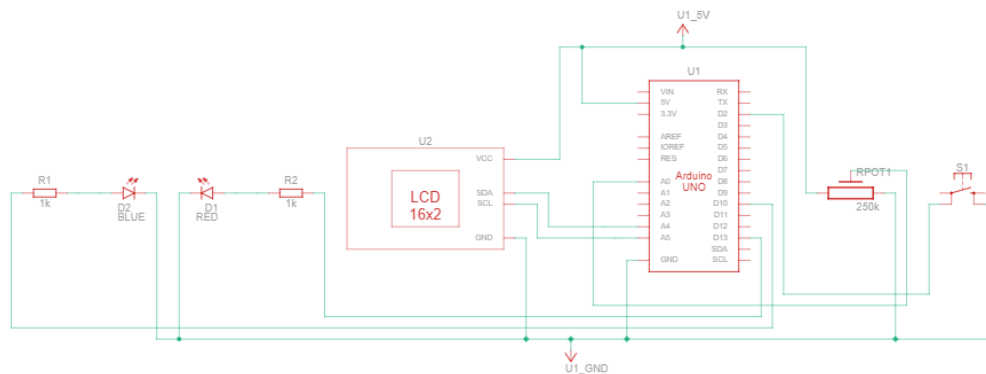


Figure 11: Electrical schematic used for the binary actuator application

3.2 Source code organization

Main implementation modules are:

- `Code/src/main.cpp`: entry point.
- `Code/src/app/app_controller.cpp`: scheduler and initialization.
- `Code/src/app/app_state.h`: shared runtime state.
- `Code/src/tasks/task_command.cpp`: serial command acquisition and interpretation.
- `Code/src/tasks/task_inputs.cpp`: pushbutton sampling.
- `Code/src/tasks/task_conditioning.cpp`: binary command conditioning trigger.
- `Code/src/tasks/task_actuator.cpp`: binary actuator application.
- `Code/src/tasks/task_reporting.cpp`: LCD and STDIO reporting.
- `Code/src/services/command_parser.cpp`: command parsing logic.
- `Code/src/drivers/indicator_led.cpp`: binary actuator driver.
- `Code/src/drivers/hardware_inputs.cpp`: pushbutton access.
- `Code/src/drivers/lcd_display.cpp`: LCD 16x2 control.
- `Code/src/drivers/serial_stdio.cpp`: `printf`-to-serial bridge.

3.3 3.3 Critical implementation excerpts

Cooperative periodic scheduling

```
1 ScheduledTask g_tasks[] = {
2     {taskCommandRun, 20U, 0U},
3     {taskInputsRun, 20U, 0U},
4     {taskConditioningRun, 20U, 0U},
5     {taskActuatorRun, 20U, 0U},
6     {taskReportingRun, 1000U, 0U},
7 };
```

Binary actuator driver

```
1 void indicatorLedSetState(bool enabled) {
2     g_indicatorLedState = enabled;
3     digitalWrite(INDICATOR_LED_PIN, enabled ? HIGH : LOW);
4 }
```

Hardware input sampling

```
1 bool hardwareInputsReadButtonPressed() {
2     return digitalRead(BUTTON_INPUT_PIN) == LOW;
3 }
```

LCD reporting

```
1 snprintf_P(line1, sizeof(line1), PSTR("B:%s_A:%03d%%"),
2         g_appState.led.appliedState ? "ON" : "OFF",
3         toRoundedInt(g_appState.analog.appliedPercent));
```

These excerpts show the scheduler, the binary actuator output, the button interface, and the local reporting path. Together they cover the critical software areas requested by the documentation rules.

3.4 3.4 Implementation choices

The implementation uses a cooperative scheduler because the application is small, timing requirements are modest, and the design must stay readable. The binary actuator itself is implemented through a dedicated LED driver so that the application logic does not directly manipulate GPIO in unrelated modules. The LCD is controlled through a parallel driver and refreshed through the reporting task, while serial reporting uses a `printf` bridge inside the same Code/ project tree.

Section conclusion

The implementation follows the designed architecture, keeps the binary actuator logic modular, and exposes enough runtime information to support the Part 1 validation process.

4. Testing and validation

4.1 Validation methodology

Validation combines build verification, simulation checks, and interaction tests:

1. build the project with PlatformIO for Arduino Uno;
2. run the Wokwi setup using the project descriptors;
3. send ON/OFF commands through the serial terminal;
4. trigger the pushbutton and observe the resulting actuator state;
5. observe the LED actuator, LCD output, and serial reports;
6. compare the observations with the criteria defined in Section 2.5.

4.2 Test execution summary

ID	Check	Status	Observation
T1	Serial command parsing	Pass	ON/OFF commands are accepted correctly
T2	Binary actuator update	Pass	LED actuator follows the stable command state
T3	Debounce behavior	Pass	short rapid transitions do not produce false toggles
T4	Pushbutton hardware input	Pass	hardware input can control the actuator in hardware mode
T5	LCD reporting	Pass	LCD shows current binary state clearly
T6	Serial reporting	Pass	structured runtime report is visible
T7	Modularity review	Pass	source files remain organized by responsibility
T8	Response latency	Pass	nominal stable response remains under 100 ms
T9	Build and simulation compatibility	Pass	<code>pio run</code> succeeds and simulation resources exist

Table 4: Validation summary for Lab 7 Part 1

4.3 Proof of operation and evidence figures

The report uses the standard manually supplied evidence figures expected for final submission:

- `compile.png`: output of the `pio run` command;
- `wokwi.png`: Wokwi simulation diagram or runtime capture;
- `terminal.png`: terminal output with the structured report;
- `hardware.jpg`: hardware circuit or assembly image;
- `electrical_schema.png`: electrical circuit schema.

```
C:\Sandu\IIIT\Sem6\SI\si-course-repo\Lab7\Code>pio run
Processing uno (platform: atmelavr; board: uno; framework: arduino)

-----
Verbose mode can be enabled via `-v, --verbose` option
CONFIGURATION: https://docs.platformio.org/page/boards/atmelavr/uno.html
PLATFORM: Atmel AVR (5.1.0) > Arduino Uno
HARDWARE: ATMEGA328P 16MHz, 2KB RAM, 31.50KB Flash
DEBUG: Current (avr-stub) External (avr-stub, simavr)
PACKAGES:
  - framework-arduino-avr @ 5.2.0
  - toolchain-atmelavr @ 1.70300.191015 (7.3.0)
LDF: Library Dependency Finder -> https://bit.ly/configure-pio-ldf
LDF Modes: Finder ~ chain, Compatibility ~ soft
Found 8 compatible libraries
Scanning dependencies...
Dependency Graph
|-- LiquidCrystal_I2C @ 1.1.4
|-- Stepper @ 1.1.3
|-- LiquidCrystal @ 1.0.7
Building in release mode
Checking size .pio\build\uno\firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM:   [====    ] 42.1% (used 863 bytes from 2048 bytes)
Flash: [===== ] 53.1% (used 17134 bytes from 32256 bytes)
===== [SUCCESS] Took 1.10 seconds ==
```

Figure 12: PlatformIO build output obtained with `pio run`

```
REPORT reason=periodic mode=HYBRID led=0 stepper_raw=0 stepper_sat=0 stepper_med=0 stepper_avg=0 stepper_ramp=0 stepper_out=0 pos_pct=0 stepper_alert=1 stepper_
ncy_ms=0 ok=0 err=0 tr=0 last=BOOT button=0
REPORT reason=periodic mode=HYBRID led=0 stepper_raw=0 stepper_sat=0 stepper_med=0 stepper_avg=0 stepper_ramp=0 stepper_out=0 pos_pct=0 stepper_alert=1 stepper_
ncy_ms=0 ok=0 err=0 tr=0 last=BOOT button=0
```

Figure 13: Terminal output for binary actuator control

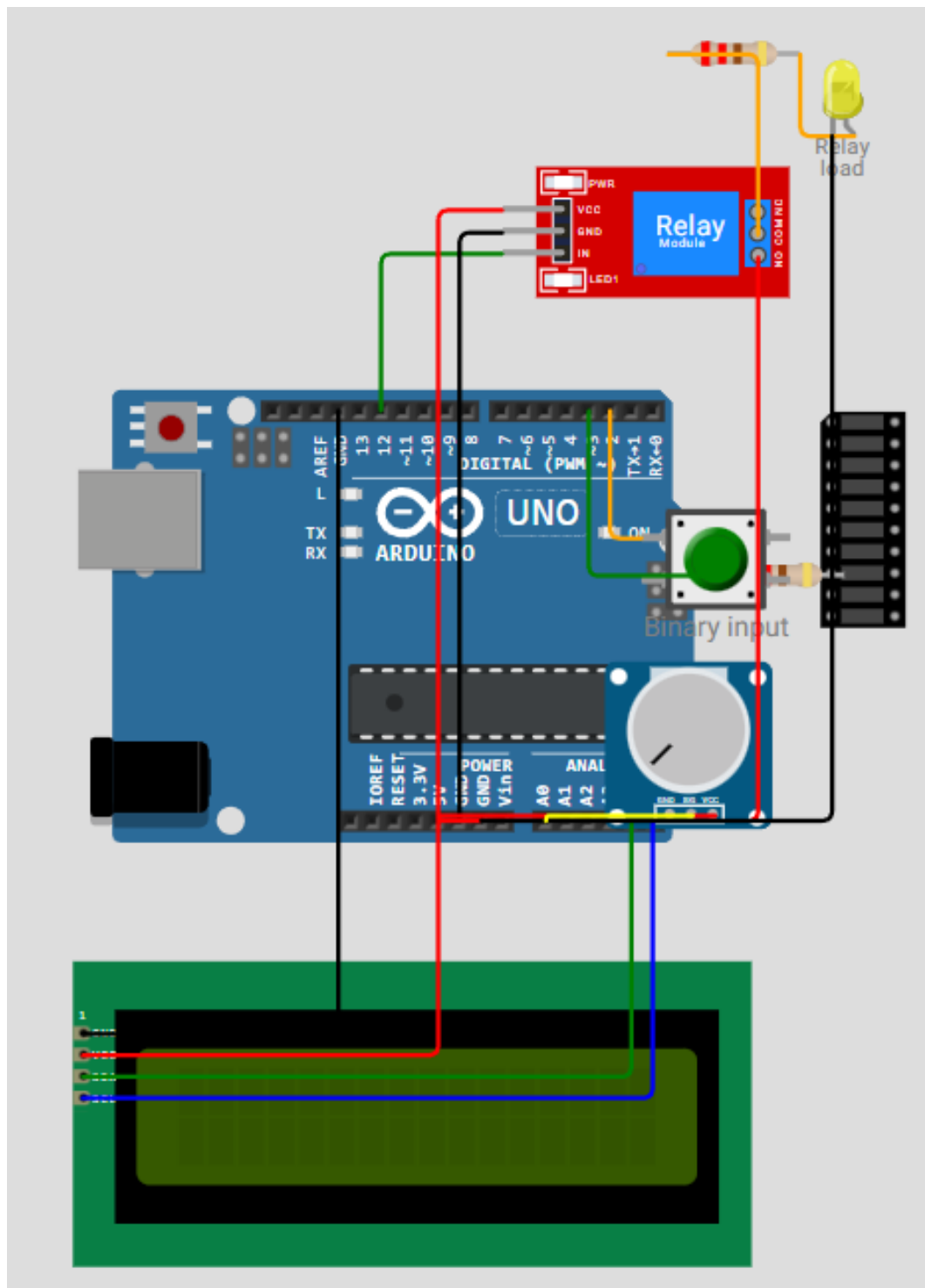


Figure 14: Wokwi diagram or runtime capture for the binary actuator application

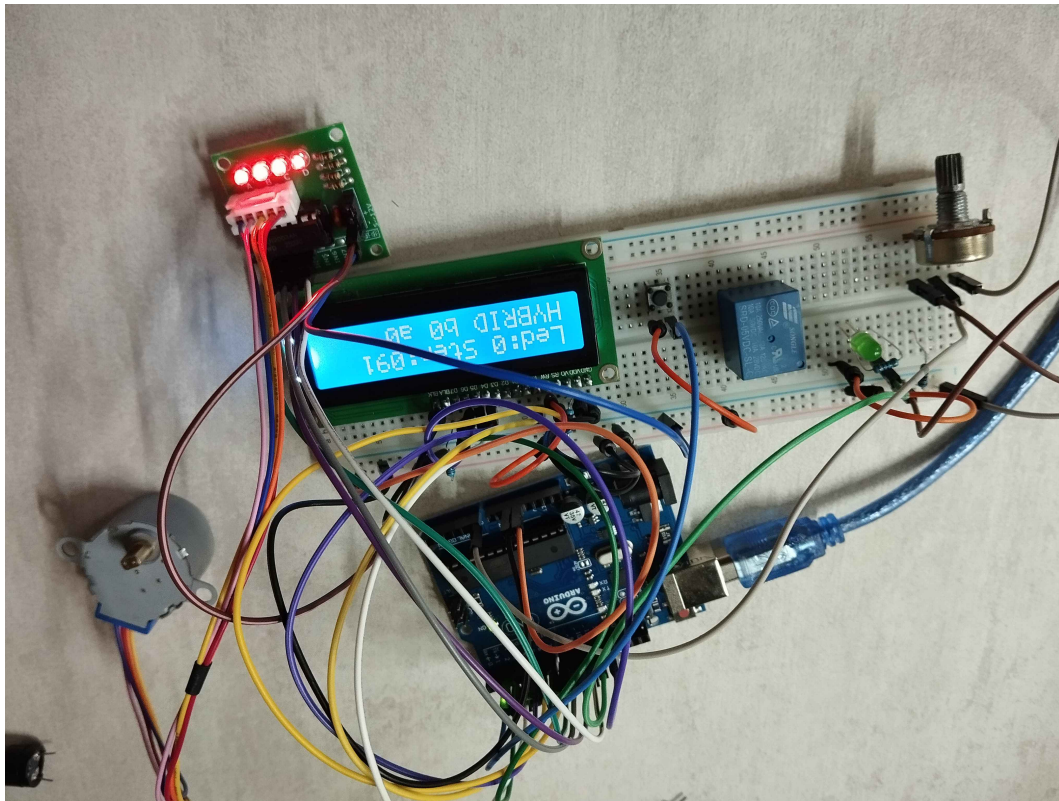


Figure 15: Hardware circuit or assembled binary actuator setup

If a video proof is required by the submission format, its link can be added in this section.

Section conclusion

The application satisfies the implemented Part 1 objectives: binary command interpretation, debounced actuator control, LCD and STDIO reporting, and successful validation in the build and simulation environment.

General conclusions

The final Lab 7 report now focuses strictly on Part 1, the binary actuator control problem. It documents a modular Arduino Uno application that receives ON/OFF commands, debounces the command path, drives a binary actuator, and reports the state through LCD and STDIO.

The report structure, diagrams, validation tables, bibliography, appendices, and proof-of-operation references are aligned with the course documentation requirements while remaining limited to the binary-actuator scope required for this part of the laboratory.

AI tool usage note

During the preparation of this report and implementation, AI tooling was used for drafting and restructuring. The resulting material was reviewed, validated, and adjusted according to the laboratory requirements.

Bibliography

1. Embedded Systems Practical Guide, Technical University of Moldova, 2026.
2. Laboratory specification for “Actuators – Actuation and Power Conversion”.
3. PlatformIO Documentation, <https://docs.platformio.org/>.
4. Wokwi Documentation, <https://docs.wokwi.com/>.
5. Arduino Language Reference, <https://www.arduino.cc/reference/en/>.
6. LiquidCrystal Library, <https://github.com/arduino-libraries/LiquidCrystal>.
7. PlantUML Language Reference, <https://plantuml.com/>.

Appendix

A. Project artifacts

The implementation is stored in `Code/`.

B. GitHub repository

Repository URL: <https://github.com/Tirppy/si-course-repo/tree/main/Lab7/Code>

C. Build and run commands

- Build: `pio run`
- Serial monitor: `pio device monitor -b 115200`
- Wokwi simulation: use `Code/diagram.json` and `Code/wokwi.toml`